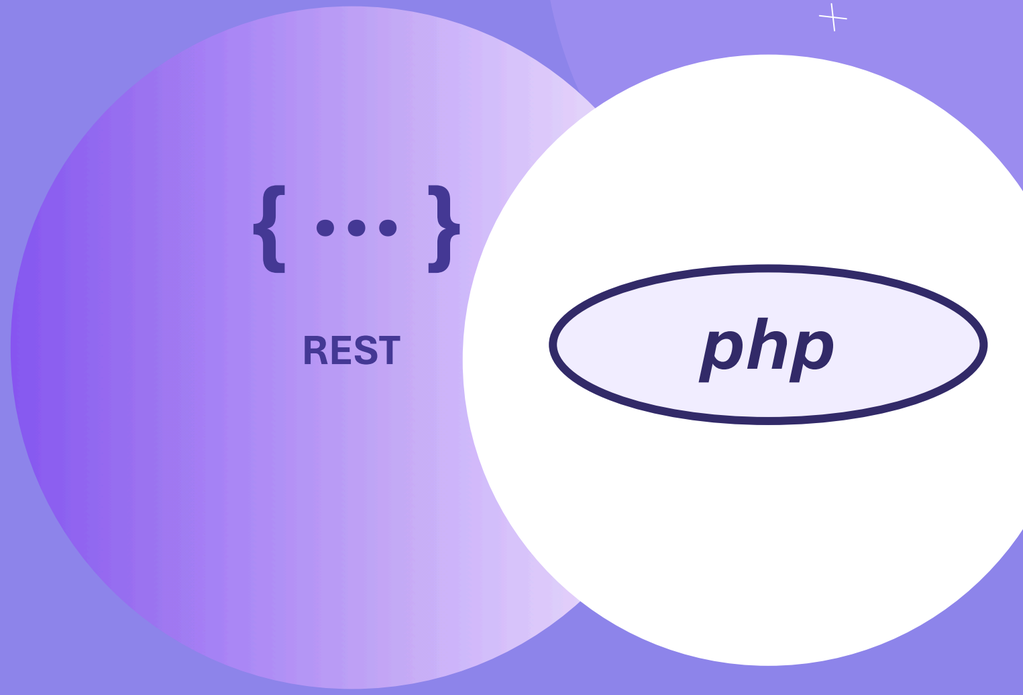
REST API in PHP:

Consumo de API REST con PHP



cURL · Guzzle · Laravel HTTP Client

SIS258 · GRUPO 4

Integrantes: Arancibia Estrada Alvaro David
Gallardo Laura Jose Manuel
Villarpando Olmedo Emily Karen

¿Cómo obtiene datos PHP de internet?

Un programa local solicita información a un servicio remoto y procesa la respuesta.

CLIENTE PHP

Construye la petición y necesita datos.

API REST

Recibe, valida y procesa la solicitud.

RESPUESTA

Datos JSON + estado HTTP.

1 · Petición

PHP identifica el recurso y utiliza un método HTTP.

2 · Procesamiento

La API ejecuta la acción correspondiente.

3 · Uso de datos

PHP decodifica JSON y muestra el resultado.

¿Qué significa consumir una API REST?

No es copiar una página web: es intercambiar peticiones y datos entre aplicaciones.

Una API REST expone recursos mediante URLs.

1. **RECURSO** — Un dato o entidad disponible, como una publicación.
2. **ENDPOINT** — Una dirección concreta, como /posts/1.
3. **SOLICITUD** — Un método HTTP indica qué operación realizar.
4. **RESPUESTA** — Código HTTP y datos estructurados, generalmente JSON.

PHP



`/posts/1`

GET

Ejemplo: consultar una publicación

Un método HTTP = una intención

La API interpreta la operación solicitada y devuelve un estado HTTP.

GET

CONSULTAR

Obtiene un recurso existente.

`GET /posts/1`

POST

CREAR

Envía datos para crear un recurso.

`POST /posts`

PUT

ACTUALIZAR

Reemplaza el contenido de un recurso.

`PUT /posts/1`

DELETE

ELIMINAR

Solicita borrar un recurso.

`DELETE /posts/1`

Estados habituales: 200 OK · 201 Created · 404 Not Found

JSON: el idioma común de la respuesta

HTTP transporta la respuesta; JSON organiza los datos que PHP utilizará.

RESPUESTA JSON · EJEMPLO ILUSTRATIVO

```
{
  "userId": 1,
  "id": 1,
  "title": "Ejemplo de publicación",
  "body": "Contenido del post"
}
```

Claves y valores

“title” es la clave; el texto es su valor.

Estructura predecible

Objetos, arreglos, números, textos y booleanos.

PHP lo interpreta así:

```
$datos = json_decode($json, true);
echo $datos['title'];
```

Misma API, tres maneras de llamarla

Lo que cambia es la cantidad de código y las herramientas utilizadas.

cURL

cURL

Extensión PHP

Permite configurar directamente cada petición.

Control manual

G

GUZZLE

Librería externa

Ofrece un cliente HTTP orientado a objetos.

Más comodidad

Http

LARAVEL

Cliente HTTP

Usa la fachada Http, construida sobre Guzzle.

Integración

Nivel de abstracción → cURL < Guzzle < Laravel HTTP Client

cURL: pedir una publicación paso a paso

Una opción flexible para controlar la petición HTTP desde PHP.

GET · $url = https://jsonplaceholder.typicode.com/posts/1

```php
<?php
$ch = curl_init($url);
curl_setopt($ch,
  CURLOPT_RETURNTRANSFER, true);
$respuesta = curl_exec($ch);
curl_close($ch);
$datos = json_decode($respuesta, true);
echo $datos['title'];
```

01 INICIAR

curl_init() crea la petición.

02 CONFIGURAR

CURLOPT_RETURNTRANSFER pide texto.

03 EJECUTAR

curl_exec() trae la respuesta.

04 LEER JSON

json_decode() crea el arreglo PHP.

Requiere la extensión cURL habilitada en PHP.

¿Y si PHP necesita enviar información?

POST incorpora datos a la petición; normalmente se envían como JSON.

POST · $url = https://jsonplaceholder.typicode.com/posts

```
$datos = ['title'=>'Hola',
  'body'=>'Mi post','userId'=>1];
$ch = curl_init($url);
curl_setopt_array($ch, [
  CURLOPT_POST => true,
  CURLOPT_RETURNTRANSFER => true,
  CURLOPT_HTTPHEADER =>
    ['Content-Type: application/json'],
  CURLOPT_POSTFIELDS =>
    json_encode($datos)
]);
```

PHP + JSON

Datos de la publicación

POST /posts

La API recibe la petición

201 Created

Creación simulada

JSONPlaceholder no guarda permanentemente los cambios.

Guzzle: hacer más con menos código

Un cliente HTTP PHP orientado a objetos que se instala con Composer.

```
composer require guzzlehttp/guzzle
```

GET · $base = https://jsonplaceholder.typicode.com/

```
require 'vendor/autoload.php';
use GuzzleHttp\Client;

$client = new Client([
 'base_uri' => $base
]);
$res = $client->get('posts/1');
$datos = json_decode($res->getBody(), true);
```

1 · Client

El objeto gestiona las solicitudes HTTP.

2 · get()

Envía una petición GET al recurso.

3 · getBody()

Recupera el cuerpo que contiene JSON.

Ventaja: menos configuración manual que con cURL.

Guzzle simplifica el POST y las respuestas

El cliente serializa JSON y proporciona métodos para consultar el estado HTTP.

POST EN GUZZLE

```
$response = $client->post('posts', [
  'json' => [
    'title' => 'Hola',
    'body' => 'Mi post',
    'userId' => 1
  ]
]);
```

```
$response->getStatusCode();
```

JSON automático

La opción json construye el cuerpo de la petición.

Respuesta HTTP

Permite consultar estado, cabeceras y contenido.

Errores

Gestionar excepciones y estados no exitosos.

Laravel HTTP Client: una llamada más directa

Su fachada Http utiliza Guzzle internamente y se integra en el framework.

GET · $url = https://jsonplaceholder.typicode.com/posts/1

```
use Illuminate\Support\Facades\Http;

$response = Http::get($url);
if ($response->successful()) {
  echo $response->json('title');
}
```

GET = recuperar información

Http::get()

Construye y envía una petición HTTP.

successful()

Comprueba si el estado está entre 200 y 299.

json("title")

Recupera una propiedad concreta del JSON.

Enviar un POST en Laravel es muy sencillo

Una estructura PHP se convierte en el cuerpo JSON de la petición.

POST · $url = https://jsonplaceholder.typicode.com/posts

```
$response = Http::post($url, [
  'title' => 'Hola',
  'body' => 'Mi post',
  'userId' => 1
]);

echo $response->status();
```

201

CREATED

JSONPlaceholder simula el registro: no lo persiste.

¿Cómo se entra a una API protegida?

Los tokens de acceso viajan en la cabecera Authorization de las solicitudes posteriores.

1

LOGIN

Credenciales por HTTPS

2

TOKEN

La API responde con un token

3

BEARER

El cliente envía el token

4

RECURSO

La API autoriza el acceso

PHP · EJEMPLO CON LARAVEL

```
$perfil = Http::withToken($token)
  ->get('https://api.ejemplo.com/perfil');
```

Nota: api.ejemplo.com es didáctico, no una API real para ejecutar.

¿Cuál deberíamos elegir para un proyecto PHP?

La mejor opción depende del framework, las dependencias y el nivel de control requerido.

<>

cURL

TIPO

Extensión PHP

CÓDIGO

Más configuración

ÚSALO PARA

Scripts sencillos y aprendizaje

G

Guzzle

TIPO

Librería Composer

CÓDIGO

Abstracción intermedia

ÚSALO PARA

Aplicaciones PHP sin Laravel

Http

Laravel

TIPO

Integrado en Laravel

CÓDIGO

Sintaxis más corta

ÚSALO PARA

Proyectos del framework

Una API, el mismo resultado, tres herramientas

Propuesta de demostración con JSONPlaceholder y el código PHP del grupo.

GET https://jsonplaceholder.typicode.com/posts/1

01 cURL

Ejecutar el script PHP y leer title.

02 Guzzle

Lanzar el cliente PHP con Composer.

03 Laravel

Probar la ruta que llama Http::get().

Comprobar: mismo endpoint → mismo JSON → mismo campo title

Repositorio del grupo: github.com/logicat1204/SIS258_Grupo4_PHP

Tres herramientas.
Una misma API.

cURL

Control explícito

Guzzle

Código mantenible

Laravel

Integración sencilla

¿Preguntas?

Fuentes: PHP Manual (cURL) · Guzzle Documentation · Laravel HTTP Client

Cuestionario

